ArtConnect Project Documentation

LESTERLIN, DELANNOY, DONNAT – PRJ Advanced Databases INT2

All the codebase is available on GitHub, at this link: [Miner205/Database-2-Project: Design and implement a relational database for the ArtConnect Java Application]

Step 1

Following the analysis of the app, we have established the following the following UML diagrams :

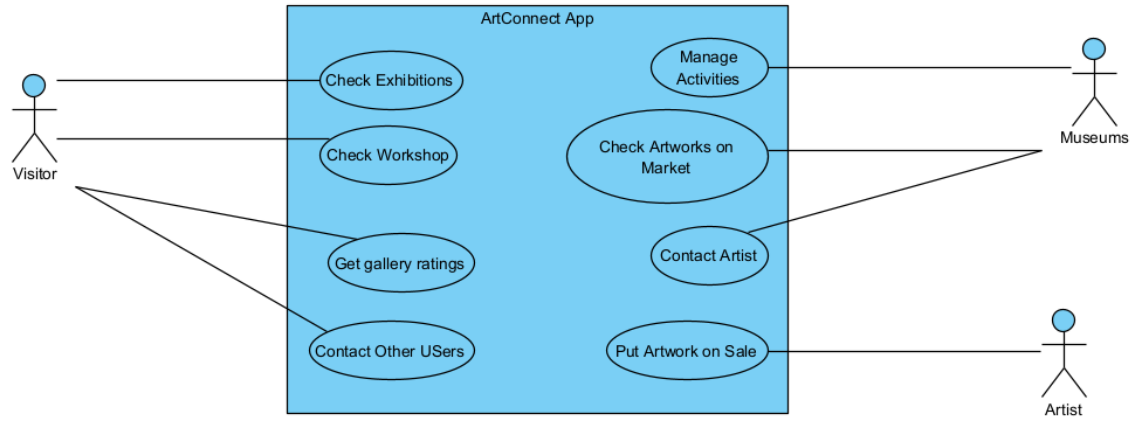


Figure 1 : Use Case Diagram of potential actors using ArtConnect.

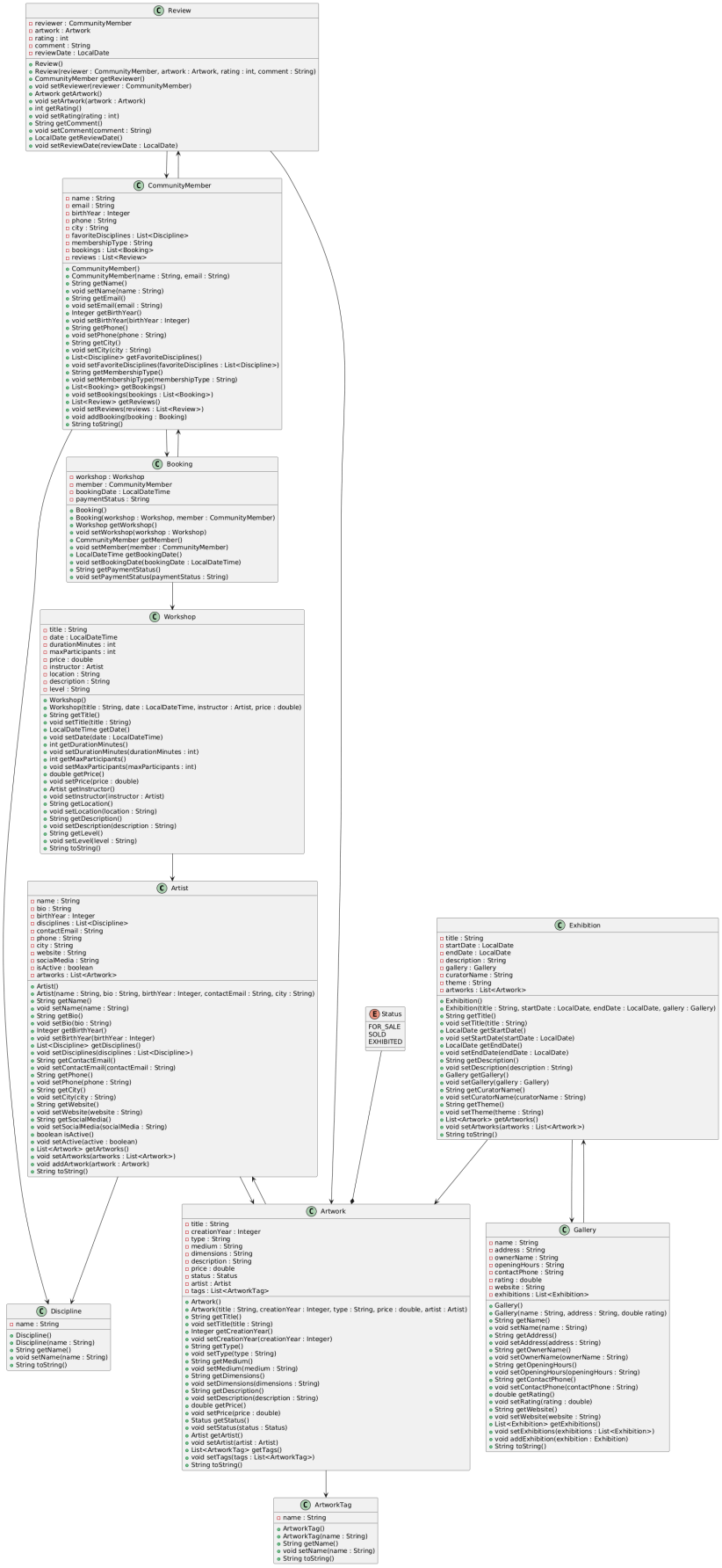


Figure 2 : Class Diagram of the currently available class structure of the project.

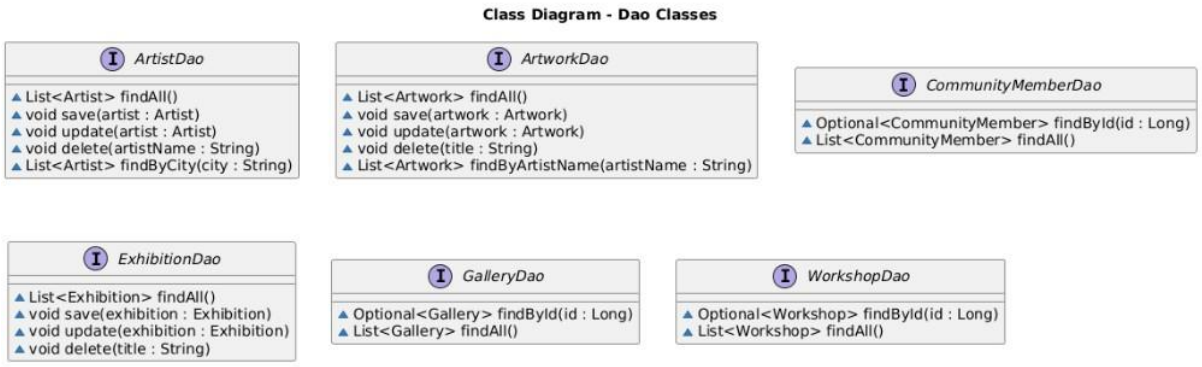


Figure 3 : Class diagram of the DAOs, our main interfaces to realize operations on the database in the Java App.

Step 2

Following this defined structure, we established the following CDM, then LDM to properly schematize the data and its relationships inside of the database. The database schematization in SQL can be retrieved from the specialized folder in the github repository.

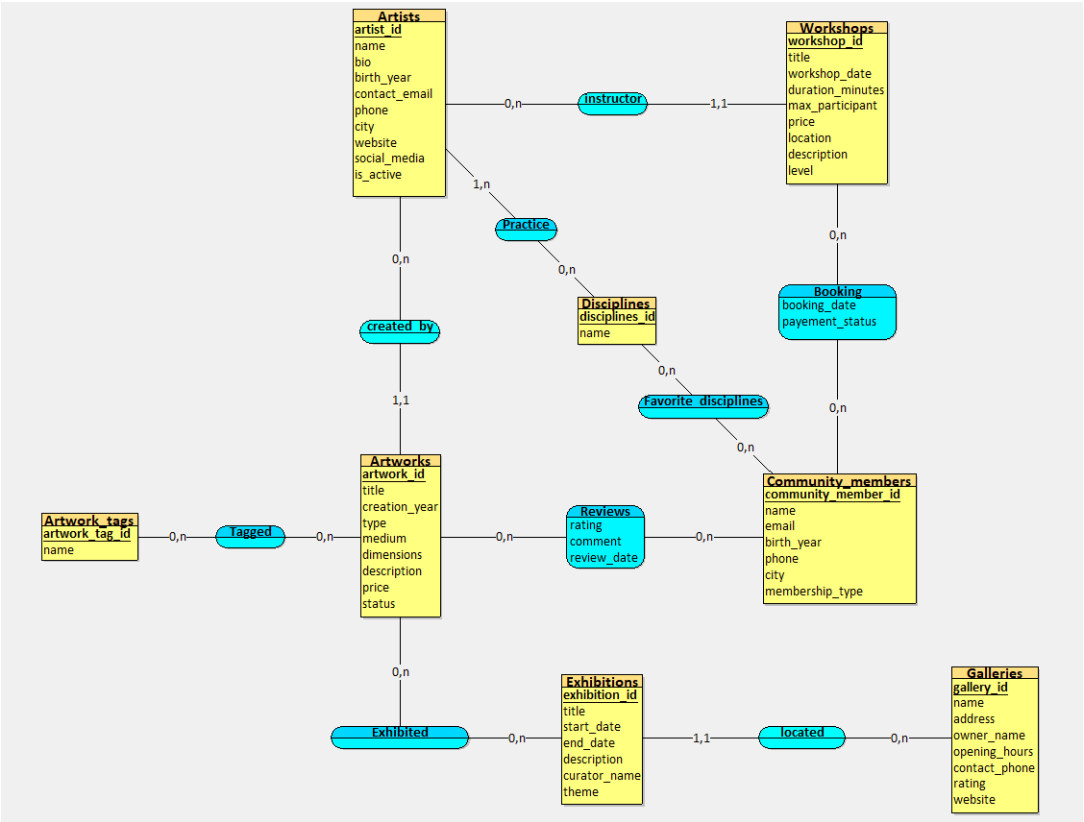
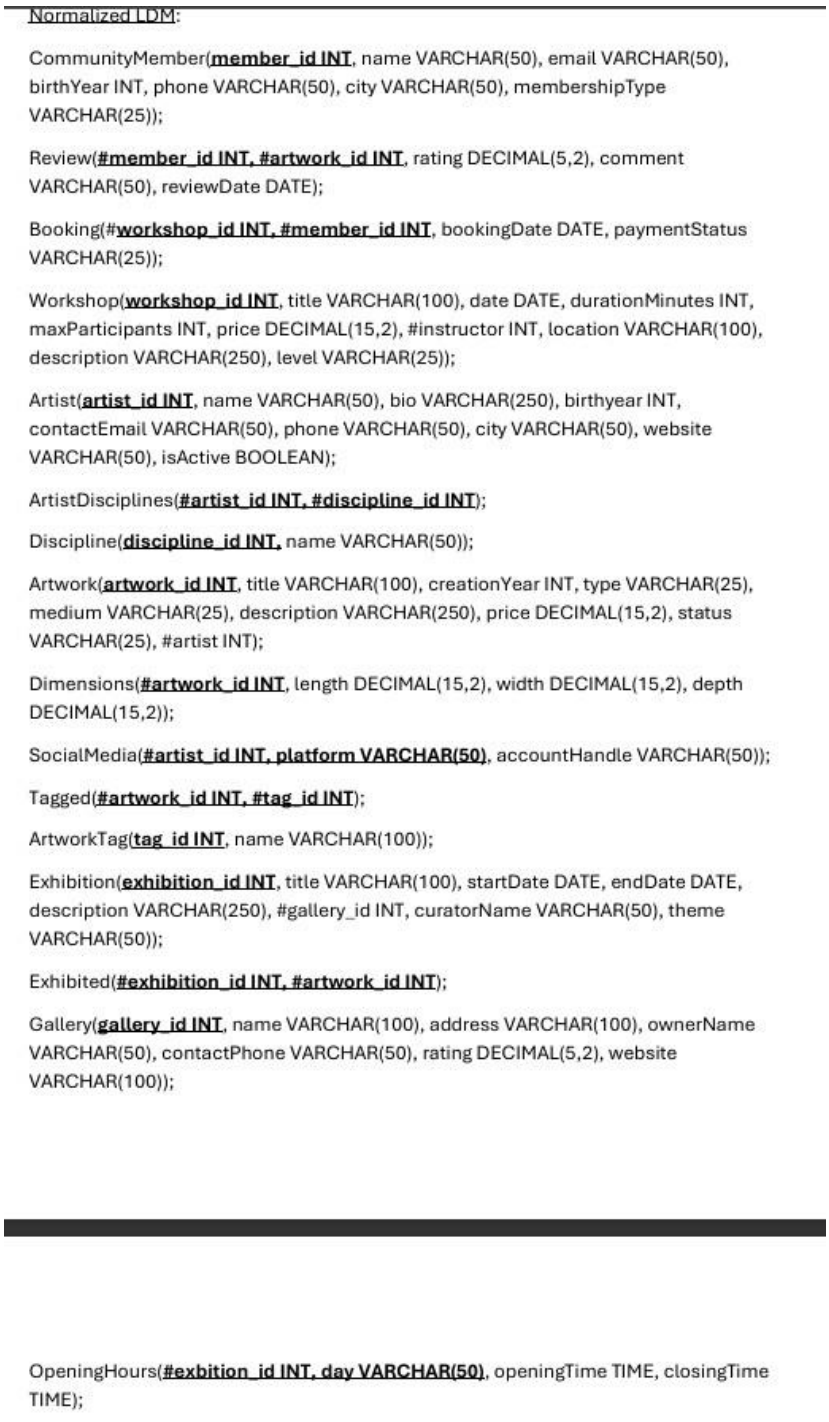


Figure 4 : CDM for the database.



Normalized LDM:

CommunityMember(**member_id INT**, name VARCHAR(50), email VARCHAR(50), birthYear INT, phone VARCHAR(50), city VARCHAR(50), membershipType VARCHAR(25));

Review(**#member_id INT, #artwork_id INT**, rating DECIMAL(5,2), comment VARCHAR(50), reviewDate DATE);

Booking(#**workshop_id INT, #member_id INT**, bookingDate DATE, paymentStatus VARCHAR(25));

Workshop(**workshop_id INT**, title VARCHAR(100), date DATE, durationMinutes INT, maxParticipants INT, price DECIMAL(15,2), #instructor INT, location VARCHAR(100), description VARCHAR(250), level VARCHAR(25));

Artist(**artist_id INT**, name VARCHAR(50), bio VARCHAR(250), birthyear INT, contactEmail VARCHAR(50), phone VARCHAR(50), city VARCHAR(50), website VARCHAR(50), isActive BOOLEAN);

ArtistDisciplines(**#artist_id INT, #discipline_id INT**);

Discipline(**discipline_id INT,** name VARCHAR(50));

Artwork(**artwork_id INT**, title VARCHAR(100), creationYear INT, type VARCHAR(25), medium VARCHAR(25), description VARCHAR(250), price DECIMAL(15,2), status VARCHAR(25), #artist INT);

Dimensions(**#artwork_id INT**, length DECIMAL(15,2), width DECIMAL(15,2), depth DECIMAL(15,2));

SocialMedia(**#artist_id INT, platform VARCHAR(50)**, accountHandle VARCHAR(50));

Tagged(**#artwork_id INT, #tag_id INT**);

ArtworkTag(**tag_id INT**, name VARCHAR(100));

Exhibition(**exhibition_id INT**, title VARCHAR(100), startDate DATE, endDate DATE, description VARCHAR(250), #gallery_id INT, curatorName VARCHAR(50), theme VARCHAR(50));

Exhibited(**#exhibition_id INT, #artwork_id INT**);

Gallery(**gallery_id INT**, name VARCHAR(100), address VARCHAR(100), ownerName VARCHAR(50), contactPhone VARCHAR(50), rating DECIMAL(5,2), website VARCHAR(100));

OpeningHours(**#exbition_id INT, day VARCHAR(50)**, openingTime TIME, closingTime TIME);

Figure 5 : LDM realized from the LDM, in 3rd NF.

Step 3

The entire code is once again available at the given github repository, containing the following :

- 4 Indexes

    - Each one indexes a “slack form” of the records of the table, I.e a very little amount of information that allows to identify more precisely artists outside of the primary key. For example, for artworks, it’s the title and creation year, for artists, the contact email, the activity and name…
- 1 Function
    - Is_Exhibited, a SQL function that returns a Boolean if an artwork is exhibited at a chosen date.
- 2 Stored Procedure
    - Echoing the function above, will display all exhibitions that displays a given artwork at a chosen date.
    - Get_nb_members_in_workshop, a sufficiently explicit name.
- 3 Triggers
    - Date_Check, ad hoc validation condition that checks if a new record has its end date after its start date;
    - Artwork_Check, a trigger that verifies if a new artwork was created after the artist’s birth year;
    - Fully_Booked, a trigger preventing a new booking of a workshop if the maximum amount of participants has been reached.
- 4 Views
    - artworks_view, gathering with the information of the artworks tables the information of the dimensions table and its tags;
    - artists_view, joining the information of the artist table with the discipline name and social media information;
    - exhibition_per_day_view, a View gathering information on an exhibition, its gallery and linked information, opening and closing hour to furnish visitors with all details;
    - members_in_workshop, a view showing all the participants of a given workshop and their payment status.
- 1 Transactional Procedure
    - Register_member_in_two_workshop, rollbacking if one of the two workshop is full.

Step 4

The ArtConnect application follows a layered architecture based on the MVC (Model–View–Controller) pattern combined with the DAO (Data Access Object) pattern.

The application is divided into several layers to ensure a clear separation of concerns:

- Presentation Layer (UI): Built with JavaFX and FXML. Controllers (e.g., ArtistController) handle user interactions and communicate with the service layer.

- Service Layer: Contains the business logic of the application (ArtistServiceImpl, ArtworkServiceImpl, etc.). Services coordinate operations between the UI and the persistence layer.
- DAO Layer: Defines DAO interfaces (ArtistDao, ArtworkDao, etc.) responsible for database operations abstraction.
- Persistence Layer: JDBC implementations (JdbcArtistDao, JdbcArtworkDao, etc.) execute SQL queries and map database data to Java objects.
- Database Layer: A MySQL database stores persistent application data such as artists, artworks, galleries, workshops, and exhibitions.

The architecture improves maintainability, modularity, and scalability by separating responsibilities between layers. It retakes design principles from a model that has been already working in modern web apps.

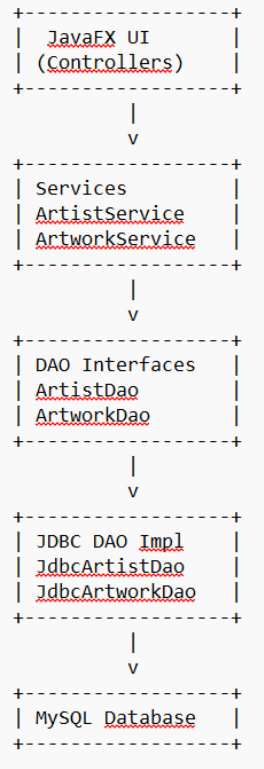


Figure 6 : Simplified class diagram of the working of the app.

Our presentation is available at this link : [Artconnect pro présentation.pptx]